Used Cars Data Platform

Comprehensive System Documentation & Technical Blueprint (Parts 1 - 4)

Attribute	Details
Initiative	Digital Egypt Pioneers Initiative (DEPI)
Track	Data Engineering
Project Title	Used Cars Data Platform (End-to-End Data Pipeline & Data Warehouse)
Team Size	4 Members

1. Project Planning & Management

1.1 Project Proposal & Executive Overview

The Used Cars Data Platform is an enterprise-grade, end-to-end data platform engineered to collect, clean, standardize, validate, store, and analyze used car listings from disparate data sources (APIs, public datasets, CSV/JSON files, and simulated streaming inputs).

In today's secondary automotive market, price transparency, valuation accuracy, and market trends analysis suffer due to fragmented data sources, inconsistent brand/model naming conventions, missing values, and unvalidated mileage or pricing figures. This project addresses these challenges by delivering an automated, batch-orchestrated Data Engineering platform utilizing Python (Pandas) for extraction/transformation, PostgreSQL for dimensional data warehousing, Apache Airflow for workflow orchestration, Docker for containerized infrastructure, and Power BI for business intelligence dashboarding.

Business Value Delivered

- **Automated Data Quality:** Guarantees that only verified, cleansed, and standardized car listings populate the analytical warehouse.
- **Centralized Intelligence:** Merges heterogeneous data sources into a unified dimensional model (Star Schema).
- **Real-time/Scheduled Insights:** Provides auto-dealerships, financial analysts, and buyers with actionable metrics on depreciation, brand retention, and price distribution.

1.2 Project Scope & Objectives

Key Objectives

- **1. Automated Ingestion:** Build a scalable ingestion pipeline capable of consuming car listing data in various formats (JSON, CSV, REST API).

- **2. Data Hygiene & Validation:** Enforce strict automated quality gates to filter out invalid records (e.g., negative prices, future manufacturing years, missing vehicle IDs).
- **3. Dimensional Data Warehousing:** Architect a normalized Staging area and an optimized Star Schema in PostgreSQL featuring Fact and Dimension tables.
- **4. Workflow Orchestration:** Schedule and monitor end-to-end DAGs (Directed Acyclic Graphs) using Apache Airflow.
- **5. Interactive Analytics:** Connect Power BI to the PostgreSQL warehouse to visualize key market metrics.

Scope Definition

In-Scope: Multi-source data ingestion (APIs, CSV, JSON); Automated data cleansing (deduplication, null handling, brand/model standardization); Data validation and quarantine tables for faulty records; Dimensional Modeling (Fact: fact_listings, Dimensions: dim_car, dim_seller, dim_location, dim_date); Dockerized containerization of PostgreSQL, Airflow, and PGAdmin; Power BI executive dashboard creation.

Out-of-Scope (Future Phase): Machine Learning model deployment for real-time price estimation (e.g., XGBoost API); Real-time streaming ingestion via Apache Kafka (project focuses on batch/scheduled ingestion).

1.3 Project Plan, Timeline & Milestones

The project timeline spans 8 weeks structured into 6 major phases following the DEPI curriculum schedule:

Phase	Task Description	Deliverable	Target Timeline
Phase 1: Initiation & Planning	Project scope definition, tech stack selection, risk assessment, and repository setup.	Project Proposal & Architecture Diagram	Week 1 - Week 2
Phase 2: Requirements & Modeling	Requirement gathering, ERD design, Data Flow Diagram (DFD) design, Star Schema layout.	SRS & ERD	Week 2 - Week 3
Phase 3: Data Ingestion & Quality	Python scripts for data collection, cleaning scripts, validation rules implementation.	Extraction & Cleaning Modules	Week 4 - Week 5
Phase 4: Warehouse & Orchestration	PostgreSQL schema execution, Airflow DAG development, Dockerizing all services.	Orchestrated ETL Pipeline in Docker	Week 5 - Week 6

Phase 5: Analytics & Power BI	Power BI connection, DAX measure creation, dashboard visual design.	Executable Power BI Dashboard	Week 7
Phase 6: Testing & Presentation	System integration testing, GitHub documentation, final presentation slide deck.	GitHub Repo & Demo Presentation	Week 8

1.4 Task Assignment & Roles (4 Team Members)

- **Team Member 1 (Lead Data Engineer):** Pipeline Orchestration (Airflow), Docker Environment Configuration, CI/CD setup.
- **Team Member 2 (Database Architect & Modeler):** PostgreSQL Schema & Star Schema Design, Data Quality Rules & Staging Tables, Database Optimization.
- **Team Member 3 (Data Ingestion & QA Specialist):** Data Scraping / API Connectors, Cleaning & Standardization Logic, QA Validation Scripting.
- **Team Member 4 (BI & Analytics Specialist):** Power BI Integration & Data Modeling, DAX Measures & Executive Dashboards, Final Documentation & Presentation.

RACI Matrix

Milestone / Task	Member 1 (Lead DE)	Member 2 (DB Arch)	Member 3 (Ingest/QA)	Member 4 (BI Spec)
Architecture & Repo Setup	Accountable	Responsible	Consulted	Informed
Data Scraping & API Ingestion	Consulted	Informed	Accountable	Informed
Data Cleaning & Validation	Responsible	Responsible	Accountable	Informed
PostgreSQL & Schema Modeling	Consulted	Accountable	Responsible	Consulted
Airflow DAG Orchestration	Accountable	Responsible	Responsible	Informed
Dockerization & Deployment	Accountable	Responsible	Informed	Informed
Power BI Dashboard & DAX	Informed	Consulted	Informed	Accountable
Documentation & QA Testing	Responsible	Responsible	Responsible	Accountable

1.5 Risk Assessment & Mitigation Plan

Risk ID	Risk Description	Impact	Likelihood	Mitigation Strategy
R-01	Schema Drift / Source API Changes: Unannounced updates to raw CSV/JSON structure breaks ingestion scripts.	High	Medium	Implement dynamic schema validation using Pydantic/Pandas schema checks before staging.
R-02	Dirty Data Overhead: Extreme outliers (e.g., price = $0 or $1,000,000) skew warehouse analytics.	High	High	Establish automated Quarantine Tables in PostgreSQL to route failed validation rows without breaking the main ETL run.
R-03	Airflow Task Failures / Container Crash: Out-of-memory errors due to processing large batches in Pandas.	Medium	Low	Limit memory allocation per Docker service, optimize Pandas chunk processing, and configure task retries in Airflow.
R-04	Power BI Connection Latency: Direct SQL queries slowing down dashboard renders.	Medium	Medium	Pre-aggregate data inside PostgreSQL Data Warehouse tables and utilize Import Mode with scheduled refreshes.

1.6 Key Performance Indicators (KPIs)

- **Data Quality Pass Rate:** >= 98% of raw records successfully cleansed and loaded into dimension/fact tables without manual intervention.
- **Pipeline SLA / Execution Time:** Complete end-to-end execution (Extract -> Transform -> Load -> Warehouse) in under 10 minutes for batch sizes up to 100,000 records.
- **System Availability & Uptime:** Airflow Orchestration and PostgreSQL Database container uptime >= 99.5%.
- **Query Latency:** Analytical queries against the Data Warehouse respond in <= 1.5 seconds.
- **Data Freshness:** Power BI dashboard refreshed within 15 minutes of new data landing in the warehouse.

2. Literature Review & Competitive Analysis

2.1 Overview of Current Used-Car Data Solutions

The digital automotive industry relies heavily on data aggregation to power pricing tools, market valuation engines, and consumer comparison websites. Current market solutions generally fall into three categories:

- **1. Static Kaggle/Public Datasets:** Frequently used for exploratory data analysis (EDA) and introductory data science projects.
- **2. Commercial Auto Aggregators (e.g., AutoTrader, CarGurus, Edmunds):** Large-scale commercial platforms offering real-time pricing and search tools.
- **3. Custom Internal ETL Scripts:** In-house scripts used by smaller dealerships to track competitors.

2.2 Strengths & Limitations of Existing Systems

Solution Category	Key Strengths	Major Limitations
Static Public Datasets	Readily accessible; Rich historical data attributes.	No Automation: One-time snapshot, quickly becomes obsolete; Lack of Quality Standards: Often contains missing IDs, uncleaned text, and duplicates.
Commercial Aggregators	Real-time live inventory; Sophisticated web UI.	Proprietary & Closed: API access is extremely restricted or costly; Rigid Schema: Users cannot inspect or customize underlying data validation pipelines.
Basic Scripting (Pandas-only)	Quick to build; Low technical barrier.	No Orchestration: Manual execution leads to failure and zero error logging; No Warehousing: Lacks a structured Star Schema for analytical tools.

2.3 Comparative Analysis & Our Unique Value Proposition

Our project bridges the gap between static datasets and proprietary systems by delivering an open-source, automated, containerized, and production-ready Data Platform built specifically for data engineering best practices.

Feature	Conventional Approaches	Our Used Cars Data Platform
Pipeline Type	Manual / One-off scripts	Fully Automated (Airflow DAGs)
Quality Control	Ad-hoc / Non-existent	Automated Quarantine Gates
Storage	Flat CSVs / Unstructured SQL	Star Schema Data Warehouse

Environment	Local environment-dependent	Fully Containerized (Docker)
Analytics	Static Notebook plots	Dynamic Power BI Dashboards

Unique Value Contributions

- **1. Automated Data Quality Isolation:** Instead of discarding bad records or letting bad data infect reporting, our validation module moves invalid records to a quarantine table while keeping the main pipeline running.
- **2. Standardization Engine:** Implements fuzzy matching and dictionary mappings to unify multi-source model names (e.g., standardizing 'BMW 3-Series', '3 Series', 'BMW 320i' under unified brand/model dimensions).
- **3. Reproducible Infrastructure:** Entire pipeline (Airflow, PostgreSQL, PGAdmin) runs via a single command (docker-compose up) ensuring zero-dependency deployment on any platform.

2.4 References & Citations

- Kimball, R., & Ross, M. (2013). 'The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling.' (3rd Edition). Wiley.
- Apache Airflow Documentation. 'Orchestrating Data Pipelines.' https://airflow.apache.org/
- Docker Inc. 'Containerization Strategies for Modern Data Stacks.' https://www.docker.com/
- DEPI Project Documentation Guidelines. Digital Egypt Pioneers Initiative, Ministry of Communications and Information Technology (MCIT), Egypt.

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder Group	Primary Interest & Need	Impact Level	System Engagement
Used Car Dealership Managers	Needs accurate market pricing trends and inventory velocity data to price cars competitively.	High	End-User (Consumes Power BI Dashboards)
Market Data Analysts	Requires clean, structured historical data for depreciation modeling and consumer behavior analysis.	High	Power BI & Direct SQL Warehouse Access
Data Engineering Team	Requires robust orchestration, automated logging, pipeline health monitoring, and data quality checks.	High	System Maintainer (Airflow & Docker)

DEPI Evaluators & Instructors	Evaluates architectural rigor, code quality, documentation completeness, and project execution.	High	Code Review & Final Presentation

3.2 User Stories & Use Cases

- **US-01 (Market Analyst):** As a market analyst, I want to view average car prices filtered by brand, model, and year, so that I can identify undervaluation and market trends.
- **US-02 (Data Engineer):** As a data engineer, I want the ingestion pipeline to automatically detect and log invalid mileage or negative prices into a quarantine table, so that raw data errors do not corrupt analytical reports.
- **US-03 (Dealership Owner):** As a dealership owner, I want to see a depreciation curve comparing Vehicle Age vs. Price drop, so that I can make smart trade-in offers.
- **US-04 (Database Administrator):** As a DBA, I want data structured in a Star Schema format, so that Power BI queries run quickly without heavy table joins.

3.3 Functional Requirements (FR)

1. Data Collection & Raw Storage

- **FR-1.1:** System shall ingest data from CSV files, JSON feeds, and mock API endpoints.
- **FR-1.2:** System shall preserve the raw, unmodified data in an immutable Staging table (staging_raw_listings) prior to applying transformations.

2. Data Cleaning & Validation

- **FR-2.1 (Deduplication):** System shall remove duplicate listings based on unique composite keys (e.g., VIN or source_id + listing_date).
- **FR-2.2 (Missing Values):** System shall impute or handle null values according to predefined business rules.
- **FR-2.3 (Standardization):** System shall standardize brand names, model names, and fuel types into unified uppercase strings.
- **FR-2.4 (Validation Rules):** System shall validate records against physical limits ($100 <= Price <= $500,000; 1990 <= Year <= Current Year; 0 <= Mileage <= 500,000 miles).
- **FR-2.5 (Quarantine Routing):** Any record failing FR-2.4 shall be written to quarantine_listings with an attached error code.

3. Transformation & Data Warehousing

- **FR-3.1:** System shall compute derived metrics, including vehicle_age (Current Year - Manufacturing Year) and price_per_mile.
- **FR-3.2:** System shall load cleansed data into a Star Schema Data Warehouse consisting of Fact Table (fact_car_listings) and Dimension Tables (dim_car, dim_location, dim_seller, dim_date).

4. Orchestration & Analytics

- **FR-4.1:** Airflow shall execute the pipeline sequentially: Ingest -> Validate -> Transform -> Load_Warehouse -> Notify.
- **FR-4.2:** Power BI shall connect to PostgreSQL to render visual charts for price trends, mileage impact, and fuel type distribution.

3.4 Non-Functional Requirements (NFR)

- **NFR-01 (Performance):** Power BI visual dashboard rendering time must remain under 2 seconds during user interactions. PostgreSQL indexing must optimize join queries.
- **NFR-02 (Scalability):** The warehouse schema and Airflow pipeline must support scaling to process over 1,000,000 listings without structural refactoring.
- **NFR-03 (Security):** Sensitive database credentials must be stored securely using environment variables (.env file) and omitted from public repositories.
- **NFR-04 (Reliability):** Airflow DAGs must feature automatic retry mechanisms (up to 3 retries with a 5-minute delay) in case of database locks or network timeouts.
- **NFR-05 (Portability):** The entire infrastructure must be reproducible via Docker containers (Docker Compose) running on Windows, macOS, or Linux.
- **NFR-06 (Maintainability):** All Python modules must follow standard formatting, modular function structure, descriptive variable names, and clear inline documentation.

4. System Analysis & Design

4.1 Problem Statement & Architectural Goals

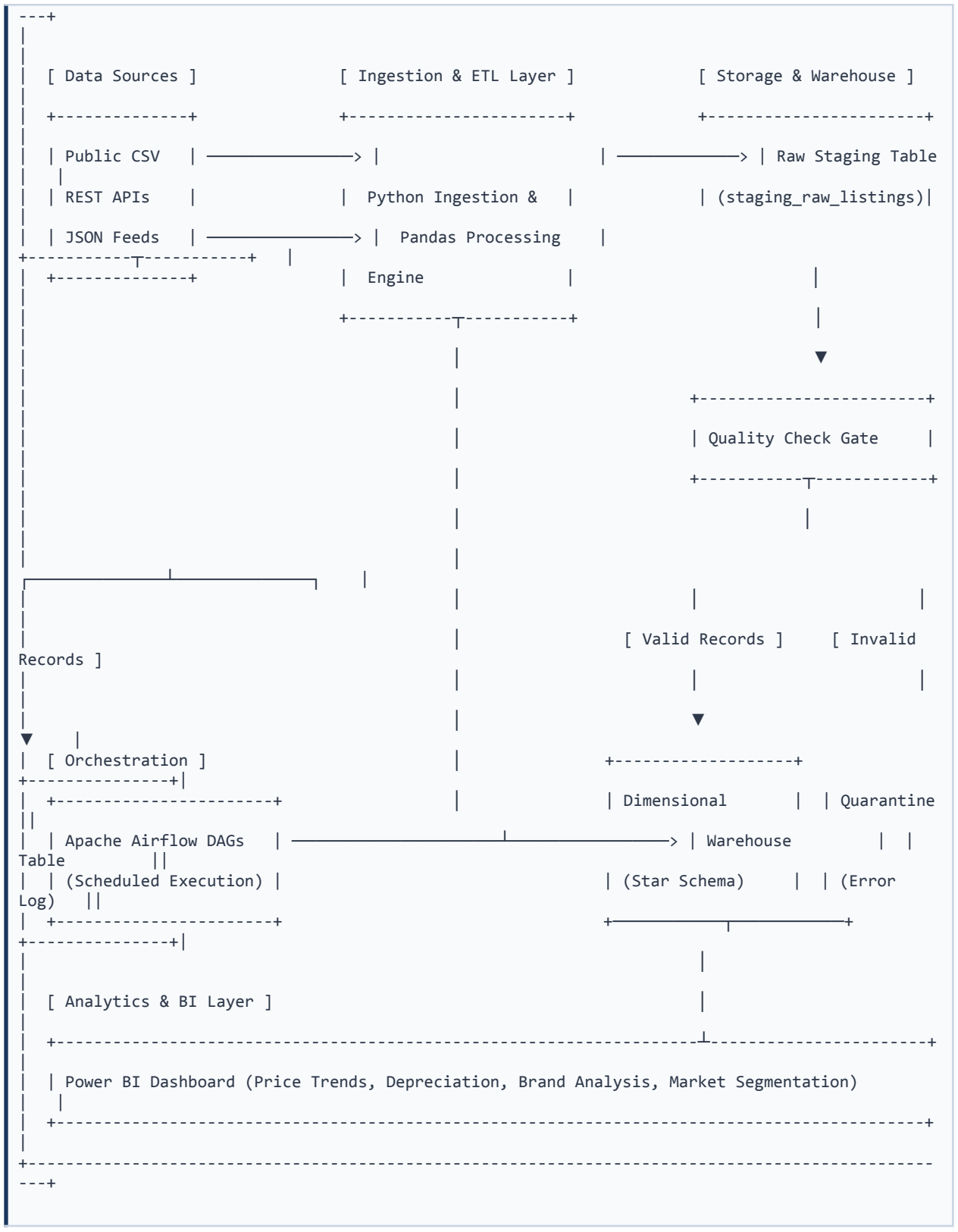
The primary problem in secondary car markets is that raw vehicle listing data is distributed across heterogeneous platforms with zero standardization. Raw feeds contain missing IDs, unformatted string representations of prices, extreme outliers, and varying brand identifiers (e.g., 'VW' vs 'Volkswagen'). Without a structured data platform, analytical queries fail or produce misleading market trends.

To solve this, the architecture is designed around four core technical goals: (1) Immutable Staging to prevent data loss, (2) Automated Quality Gatekeeper to isolate corrupted records into Quarantine, (3) Dimensional Star Schema to optimize analytical query performance for Power BI, and (4) Modular Orchestration & Containerization to ensure environment reproducibility.

4.2 Software Architecture & High-Level Design

The system follows a modern batch ETL Data Engineering architecture fully containerized using Docker Compose. Below is the end-to-end component interaction flow:

```
+-------------------------------------------------------------------------------------------------
---+
|                                HIGH-LEVEL SYSTEM ARCHITECTURE
|
+-------------------------------------------------------------------------------------------------
```


[Data Sources]
[Ingestion & ETL Layer]
[Storage & Warehouse]
Public CSV
REST APIs
JSON Feeds
Python Ingestion & Pandas Processing Engine
Raw Staging Table (staging_raw_listings)
Quality Check Gate
[Valid Records]
[Invalid Records]
[Orchestration]
Apache Airflow DAGs (Scheduled Execution)
Dimensional Warehouse (Star Schema)
Quarantine Table (Error Log)
[Analytics & BI Layer]
Power BI Dashboard (Price Trends, Depreciation, Brand Analysis, Market Segmentation)

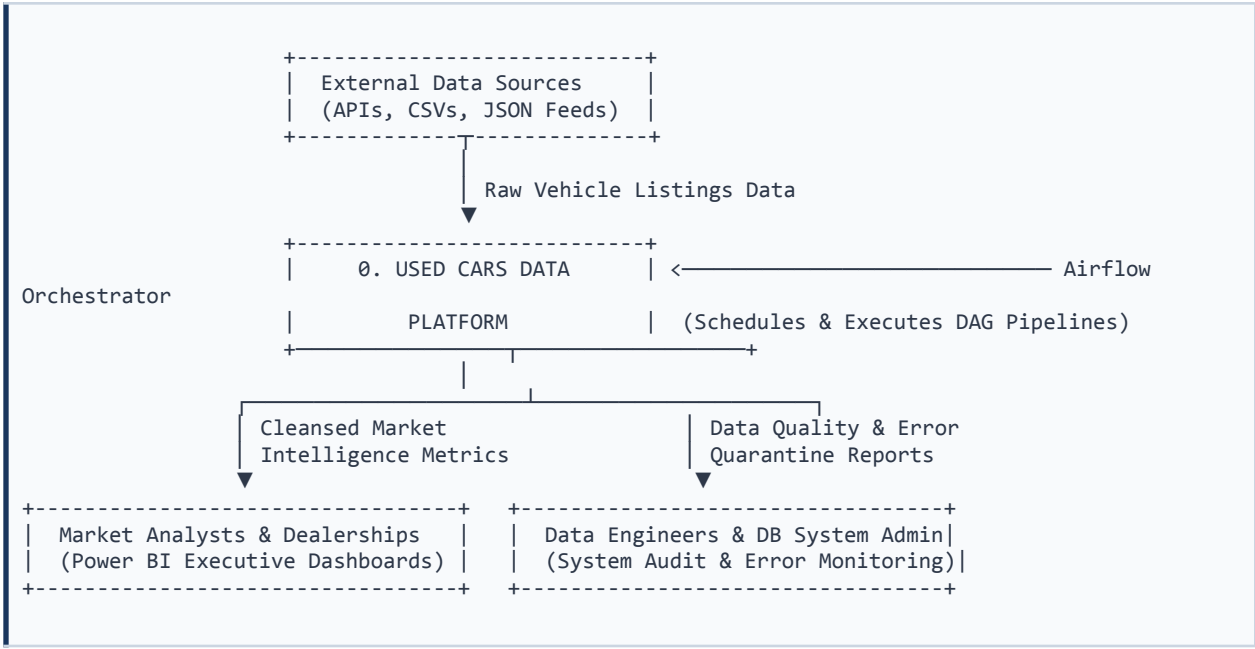
Component Breakdown

- **1. Extraction Module (Python/Pandas):** Fetches records from API endpoints, JSON payloads, and CSV drops, mapping them to uniform Pandas DataFrames.
- **2. Staging Database (PostgreSQL):** Stores unmodified raw data inside PostgreSQL staging_raw_listings for auditability.
- **3. Validation & Quarantine Engine:** Validates data against strict rules (ranges, null checks, duplicate keys). Moves bad rows to quarantine_listings and clean rows to transformation.
- **4. Dimensional Data Warehouse:** Structures cleaned records into Star Schema dimensional tables optimized for star joins and aggregation.
- **5. Airflow Orchestration:** Schedules daily/hourly DAG runs, manages task dependencies, triggers retries, and logs errors.
- **6. Power BI Visualization:** Imports data from PostgreSQL Star Schema to generate dynamic visual reports.

4.3 Data Flow Diagrams (DFD)

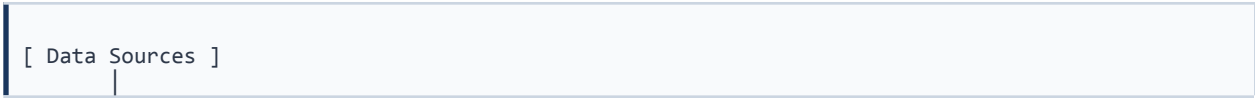
DFD Level 0 (Context Diagram)

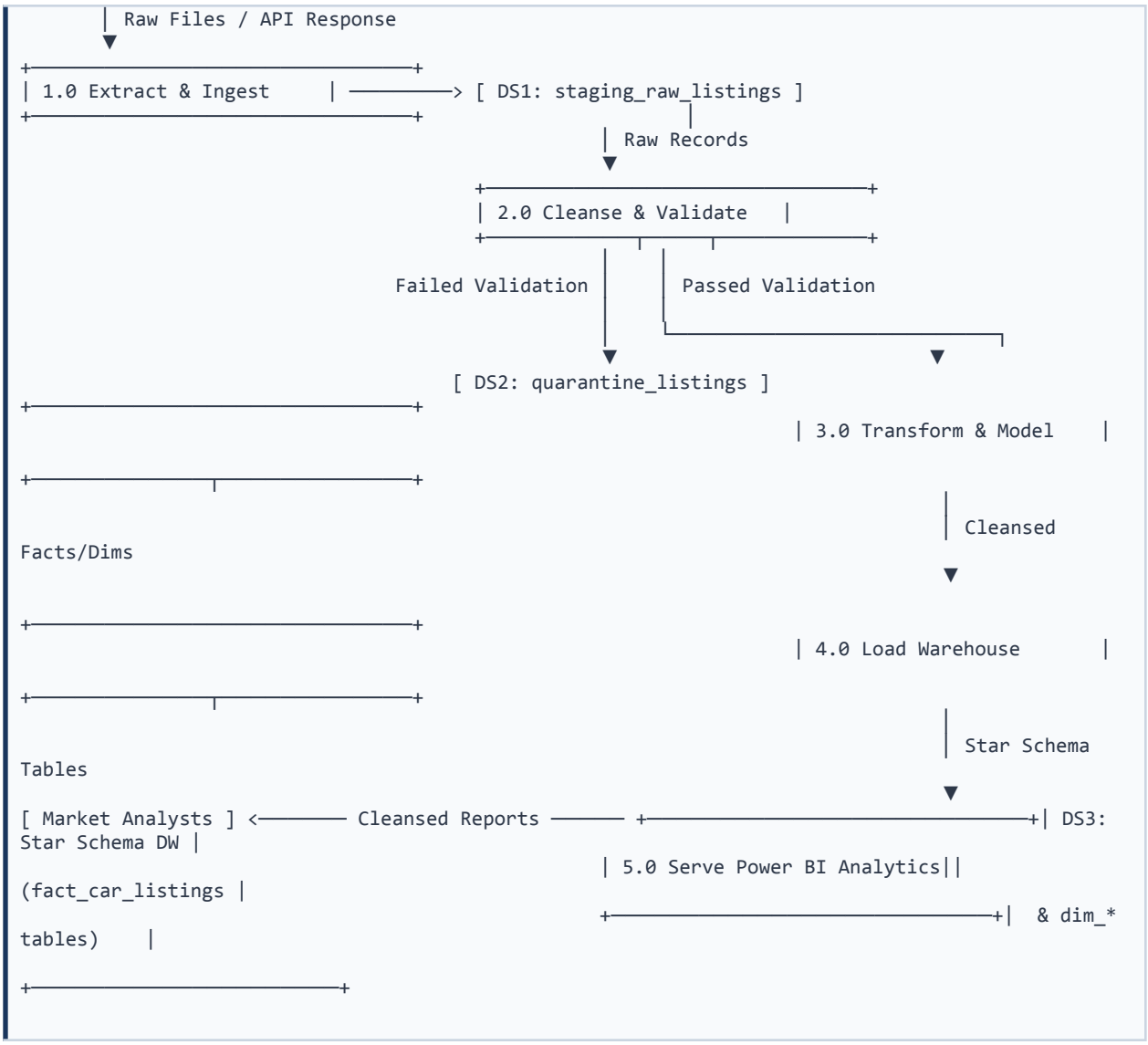
The Context Diagram illustrates the boundaries of the Used Cars Data Platform, highlighting interactions with external entities:



DFD Level 1 (Detailed Data Flow Diagram)

DFD Level 1 decomposes the system into 5 core subprocesses and 3 primary data stores:





4.4 Entity-Relationship Diagram (ERD) & Staging Schemas

Prior to loading into the Data Warehouse Star Schema, raw records enter the Staging and Quarantine tables. Below are the physical schema specifications for the ingestion layer:

1. Staging Table: staging_raw_listings

Column Name	Data Type	Constraint / Key	Nullable	Description
raw_id	BIGSERIAL	PRIMARY KEY	No	Auto-incrementing surrogate primary key for staging.
source_system	VARCHAR(50)	None	No	Identifier of the data source (e.g.,

				'API_AUTOTRADER', 'CSV_BATCH_1').
source_listing_id	VARCHAR(100)	None	No	Original ID provided by the external source.
vin	VARCHAR(17)	None	Yes	Vehicle Identification Number (17 chars).
raw_title	TEXT	None	Yes	Uncleaned listing title string.
brand_raw	VARCHAR(100)	None	Yes	Raw brand name (e.g., 'bmw', 'B.M.W.').
model_raw	VARCHAR(100)	None	Yes	Raw model name (e.g., '3 series', '320i').
year_raw	VARCHAR(20)	None	Yes	Unparsed manufacturing year.
price_raw	VARCHAR(50)	None	Yes	Unparsed price string (e.g., '$15,000 USD').
mileage_raw	VARCHAR(50)	None	Yes	Unparsed odometer reading.
fuel_type_raw	VARCHAR(50)	None	Yes	Raw fuel type string.
seller_name_raw	VARCHAR(150)	None	Yes	Raw seller name.
location_raw	VARCHAR(150)	None	Yes	Raw city/state/zip string.
ingested_at	TIMESTAMP	DEFAULT NOW()	No	System timestamp when row landed in staging.

2. Quarantine Table: quarantine_listings

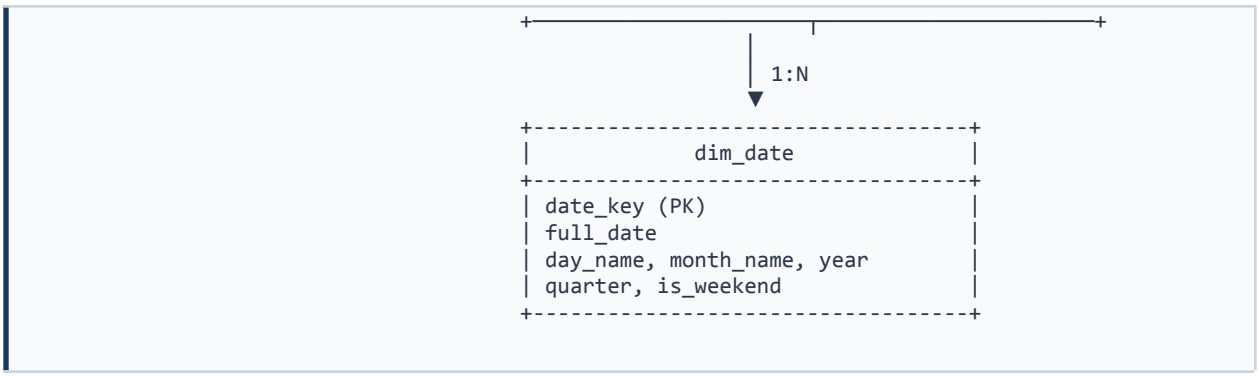
Column Name	Data Type	Constraint / Key	Nullable	Description
quarantine_id	BIGSERIAL	PRIMARY KEY	No	Unique quarantine log entry identifier.
raw_id	BIGINT	FOREIGN KEY	Yes	References staging_raw_listings.raw_id.
error_code	VARCHAR(50)	None	No	Standard error classification (e.g.,

				'ERR_INVALID_PRICE').
error_message	TEXT	None	No	Detailed explanation of validation rule failure.
failed_field	VARCHAR(50)	None	No	Field that triggered the failure.
failed_value	TEXT	None	Yes	Actual value that failed validation.
quarantined_at	TIMESTAMP	DEFAULT NOW()	No	Timestamp when entry was rejected.

4.5 Dimensional Modeling (Star Schema Design)

To maximize query responsiveness for analytical workloads in Power BI, the Data Warehouse is structured as a Star Schema. It centers around one core Fact table surrounded by four conforming Dimension tables:





1. Fact Table: fact_car_listings

Column Name	Data Type	Constraint / Key	Description
fact_id	BIGSERIAL	PRIMARY KEY	Surrogate primary key for the listing fact record.
car_key	INT	FOREIGN KEY -> dim_car	Surrogate key referencing car dimension.
seller_key	INT	FOREIGN KEY -> dim_seller	Surrogate key referencing seller dimension.
location_key	INT	FOREIGN KEY -> dim_location	Surrogate key referencing location dimension.
date_key	INT	FOREIGN KEY -> dim_date	Surrogate key referencing date dimension (YYYYMMDD).
listing_id	VARCHAR(100)	Degenerate Dim	Natural key from source for transactional tracking.
price	NUMERIC(12,2)	Measure (Additive)	Final validated selling price in USD.
mileage_miles	INT	Measure (Additive)	Validated odometer reading in miles.
vehicle_age_years	INT	Measure (Additive)	Computed age (Current Year - Manufacturing Year).
price_per_mile	NUMERIC(10,4)	Measure (Semi-Add)	Calculated ratio: Price / Mileage.
days_on_market	INT	Measure (Additive)	Number of days listing has been active.

2. Dimension Table: dim_car

Column Name	Data Type	Constraint / Key	Description

car_key	SERIAL	PRIMARY KEY	Surrogate key for car entity.
vin	VARCHAR(17)	UNIQUE / Natural Key	Standardized 17-character VIN.
brand_name	VARCHAR(50)	None	Standardized brand (e.g., 'BMW', 'TOYOTA').
model_name	VARCHAR(50)	None	Standardized model name (e.g., '3 SERIES', 'CAMRY').
body_style	VARCHAR(30)	None	Sedan, SUV, Coupe, Truck, Convertible, etc.
transmission_type	VARCHAR(20)	None	Automatic, Manual, CVT, Dual-Clutch.
fuel_type	VARCHAR(20)	None	Gasoline, Diesel, Electric, Hybrid.
engine_capacity_cc	INT	None	Engine displacement in cubic centimeters.

3. Dimension Tables: dim_seller, dim_location, dim_date

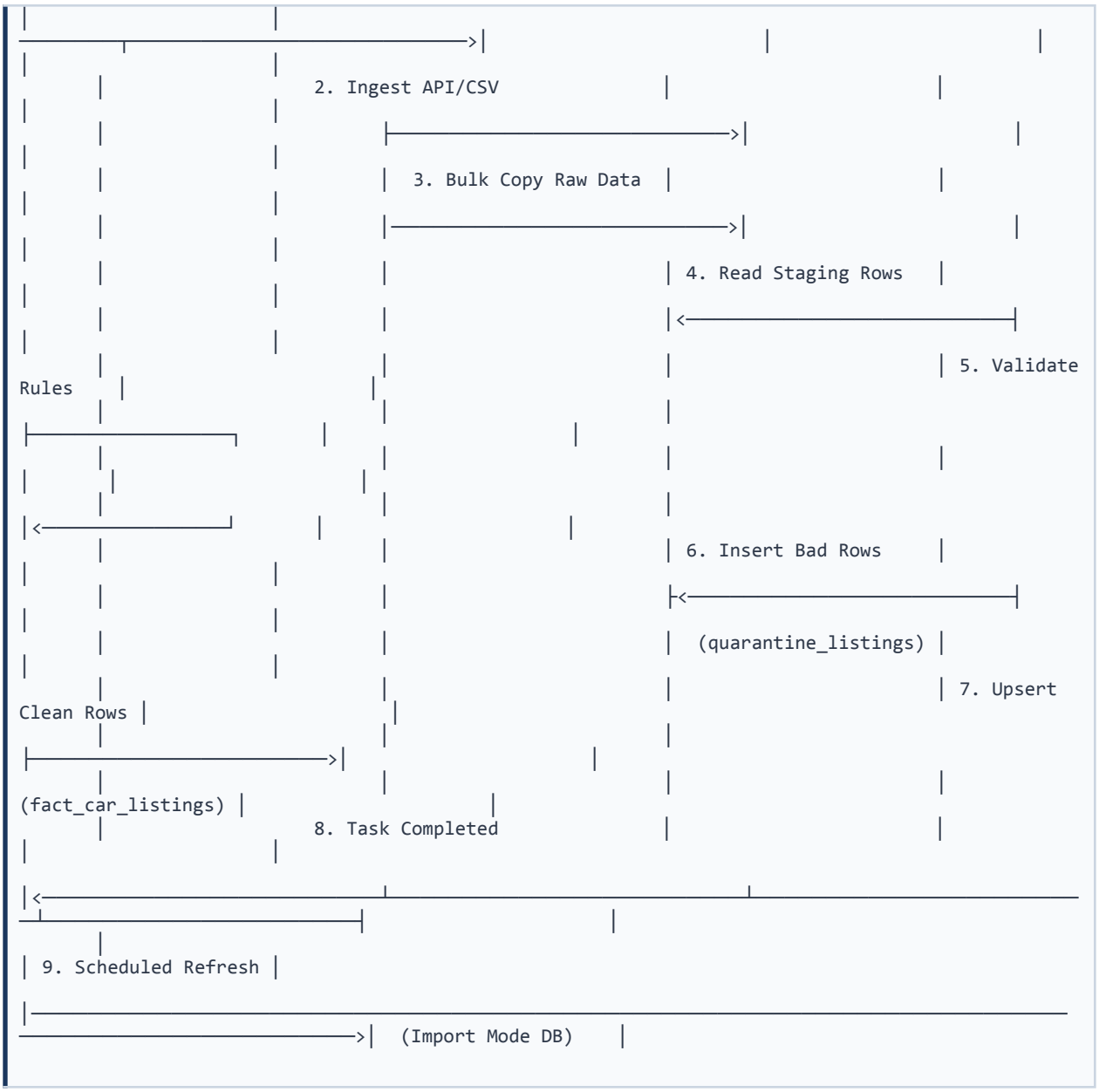
Dimension Table	Surrogate PK	Key Attributes	Business Purpose
dim_seller	seller_key (INT)	seller_id, seller_name, seller_type, seller_rating	Tracks dealership vs private sellers and performance ratings.
dim_location	location_key (INT)	zip_code, city, state, region	Enables spatial analysis of pricing variations across geography.
dim_date	date_key (INT)	full_date, day_of_week, month, quarter, year, is_weekend	Supports time-series trends, seasonality, and time intelligence DAX.

4.6 System Behavior Diagrams

Sequence Diagram (End-to-End Execution Flow)

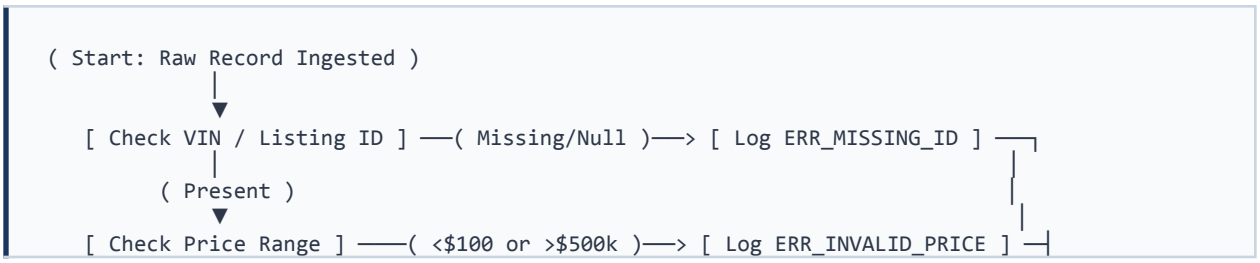
The Sequence Diagram details step-by-step execution across components during a scheduled pipeline trigger:

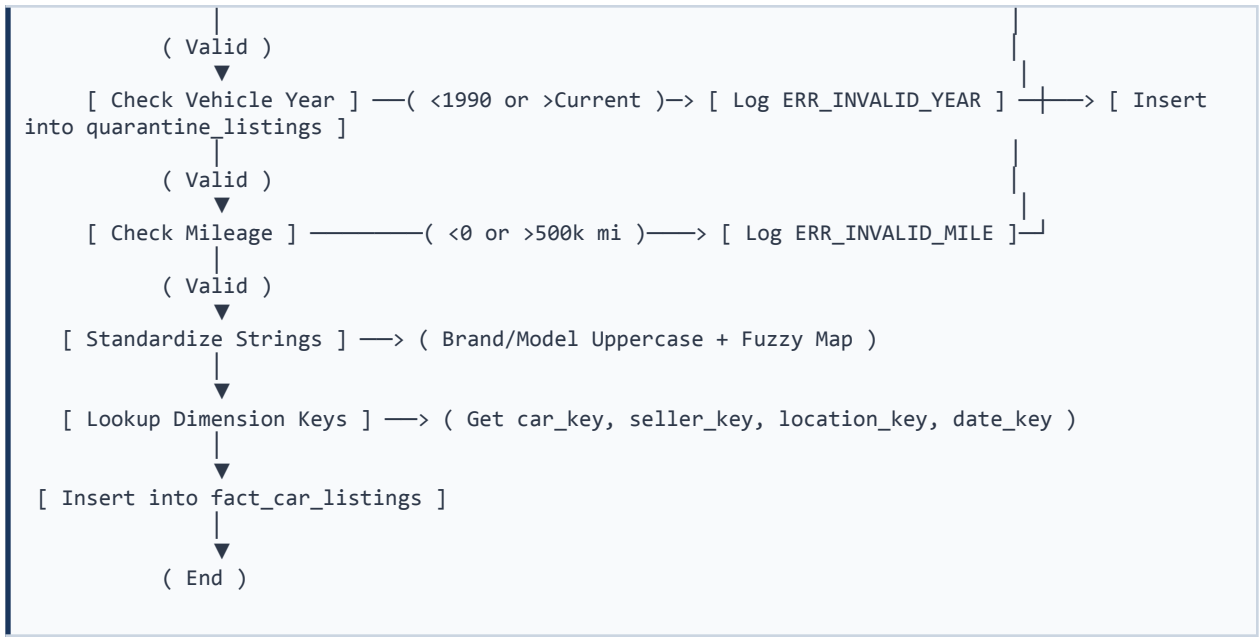
```
Airflow Scheduler     Python Pipeline Engine     PostgreSQL Staging DB     Quality Gate Engine
PostgreSQL Star DW      Power BI Service
       |                        |                          |                        |
|                    |
  1. Trigger DAG                |                          |                        |
```



Activity Diagram (Data Validation Logic Flow)

The Activity Diagram illustrates decision paths applied to each incoming record by the Data Quality Gatekeeper:

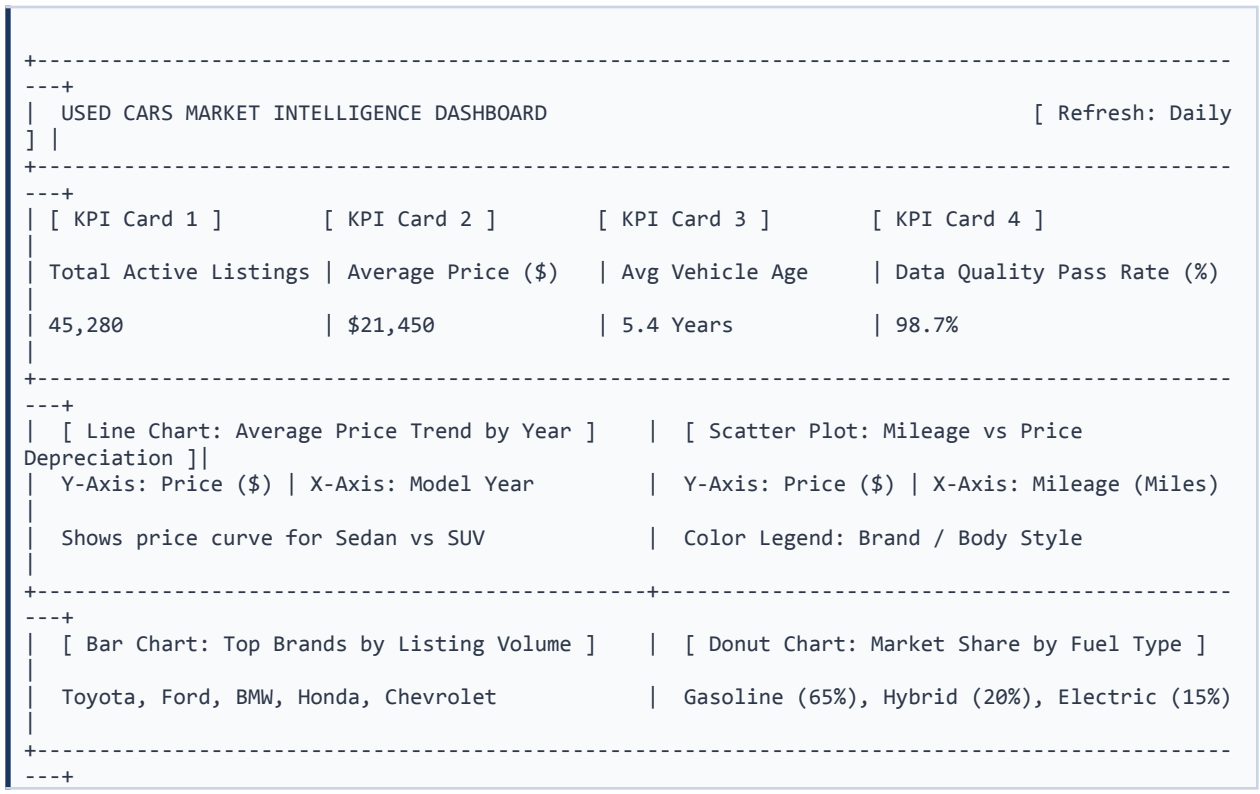




4.7 Power BI UI/UX Wireframe & DAX Measures

Dashboard Wireframe Layout

The visual dashboard is structured into an executive grid providing overview KPIs, price trend analysis, mileage depreciation curves, and brand segmentation:



```
|  [ Slicers / Filters Bar ]: [ Region: All ] | [ Brand: All ] | [ Year Range: 2015-2024 ]
|
+-------------------------------------------------------------------------------------------
---+
```

Key DAX Calculated Measures

Measure Name	DAX Formula Expression	Business Objective
Average Price	AveragePrice = AVERAGE(fact_car_listings[price])	Calculates mean market price filtered by visual slicers.
Price per Mile Ratio	PricePerMile = DIVIDE(SUM(fact_car_listings[price]), SUM(fact_car_listings[mileage_miles]), 0)	Evaluates value density per mile across vehicle categories.
Data Quality Pass Rate	QualityPassRate = DIVIDE(COUNT(fact_car_listings[fact_id]), COUNT(fact_car_listings[fact_id]) + COUNT(quarantine_listings[quarantine_id]), 0)	Monitors pipeline health and data clean pass percentage.
Depreciation Rate %	DepreciationRate = DIVIDE([Average Price] - CALCULATE([Average Price], dim_date[year] = YEAR(TODAY()) - 5), [Average Price], 0)	Quantifies 5-year value depreciation percentage for trade-in estimation.

4.8 System Deployment & Docker Integration

The infrastructure is containerized using Docker Compose, guaranteeing seamless cross-platform execution with a single command (`docker-compose up -d`):

Service Container	Base Image	Exposed Port	Role & Responsibility
airflow-webserver	apache/airflow:2.7.1-python3.10	8080:8080	Airflow UI dashboard for DAG monitoring and trigger management.
airflow-scheduler	apache/airflow:2.7.1-python3.10	None (Internal)	Monitors DAG tasks and triggers execution workers based on schedule.
postgres-dw	postgres:15-alpine	5432:5432	Hosts staging_raw_listings, quarantine_listings, and Star Schema DW.
pgadmin	dpage/pgadmin4:latest	5050:80	Web-based administration tool for SQL query

	execution and database inspection.